

Spreadsheet Formula Language Front-End Report

1. Project Overview

This project implements a robust compiler front-end designed to process spreadsheet-like formulas. The system is built using a **handwritten scanner (lexer)** and a **handwritten recursive-descent parser** in Java.

Key Deliverables:

- Handwritten Lexical Analyzer (Scanner).
- Recursive-Descent Parser.
- Abstract Syntax Tree (AST) Construction.
- Comprehensive Syntax and Semantic Error Reporting.

2. Language Scope & Features

2.1 Required Features

- **Entry Point:** All formulas must begin with the = symbol.
- **Literals:** Support for integer literals.
- **References:** Cell references (e.g., A1, B2, AA10).
- **Operators:** Arithmetic operators (+, -, *, /) with proper precedence.
- **Functions:** Implementation of SUM, MAX, MIN, and IF.
- **Structure:** Parentheses for grouping and comma-separated function arguments.

2.2 Optional Extensions

- **Ranges:** Support for cell ranges like A1:A5.
- **Nesting:** Full support for nested function calls.
- **Export:** AST export functionality in **JSON** format.
- **Batch Processing:** Formula file mode (one formula per line).

3. Grammar Description (EBNF)

The grammar is designed to be **Recursive-Descent friendly**, handling precedence from comparison down to primary expressions:

EBNF

```
formula    ::= "=" expression EOF ;
expression ::= comparison ;
comparison ::= additive ( ">" | "<" | ">=" | "<=" | "==" | "!=" ) additive)* ;
additive   ::= multiplicative ( "+" | "-" ) multiplicative)* ;
multiplicative ::= primary ( "*" | "/" ) primary)* ;
primary    ::= NUMBER | CELL_REF [ "." CELL_REF ] | function_call | "(" expression ")" ;
function_call ::= FUNCTION "(" [ argument_list ] ")" ;
argument_list ::= expression ( "," expression)* ;
```

4. Technical Implementation

4.1 Scanner / Lexical Analysis

- **Tokenization:** Converts input strings into a stream of tokens (e.g., NUMBER, CELL_REF, FUNCTION).
- **Lexeme Storage:** Distinctly stores both the token type and the literal lexeme value.
- **Normalization:** Function names are automatically normalized to uppercase to prevent case-sensitivity issues.

4.2 Parser / Structural Output

- **Logic:** Parser procedures directly reflect the grammar's nonterminals.
- **AST Design:** Nodes like FormulaNode, BinaryNode, and FuncNode represent the code structure rather than a simple parse trace.
- **Error Handling:** Reports lexical errors (invalid chars) and syntax errors (missing parentheses/commas) without crashing.

5. Testing & Diagnostics

5.1 Valid Examples

- `=SUM(A1:A5, 10)` (Range and Literal)
- `=IF(A1 > 10, B1, 0)` (Conditional Logic)
- `=MAX(SUM(A1,2), MIN(B1,3))` (Nested Functions)

5.2 Error Handling (Invalid Cases)

- `A1 + B2`: Rejected (Missing `=`).
- `=SUM(A1 B2)`: Rejected (Missing comma).
- `=A5:A1`: Semantic check for malformed descending range.

6. Milestone Supervision Mapping

The project has successfully met all criteria defined in the **Supervision Handout**:

- **Milestone 1-2**: Language defined and scanner independently verified.
- **Milestone 3-4**: Core parser implemented with stable AST and diagnostics.
- **Milestone 5**: Final demo readiness, including JSON output and technical justifications.

7. Design Choices & Limitations

- **Choice**: Function arguments are parsed as full expressions, allowing for infinite nesting and rich formula complexity.
- **Limitation**: The current version does not support absolute references (e.g., `$A$1`) or cross-sheet references.